Java Topics to Revise for TCS NQT

For **TCS NQT (Java)** the questions usually focus on **core language concepts, standard library usage, and basic problem solving**. Based on previous NQT patterns, these are the **most important Java topics to revise**.

Focus on these first:

1. **String methods**
2. **Array operations**
3. **ArrayList**
4. **HashMap**
5. **HashSet**
6. **Arrays.sort()**
7. **StringBuilder**
8. **Exception handling**
9. **Wrapper classes**
10. **OOP basics**

1. Important Java Methods (Math & Utility)

These are frequently used in coding questions.

Important Math methods:

- Math.abs()
- Math.max()
- Math.min()
- Math.pow()
- Math.sqrt()
- Math.floor()
- Math.ceil()
- Math.round()

Example:

```
int a = 10;
int b = -5;

System.out.println(Math.abs(b));   // 5
System.out.println(Math.max(a,b)); // 10
System.out.println(Math.pow(2,3)); // 8.0
```

2. Wrapper Classes

Primitive types converted into objects.

Common wrapper classes:

- Integer
- Double
- Character
- Boolean

Example:

```
Integer x = 10;

System.out.println(Integer.parseInt("20"));
System.out.println(Integer.valueOf("30"));
```

Important methods:

- parseInt()
- valueOf()
- toString()

3. String Methods (Very Important)

Very frequently asked in **MCQ + output prediction**.

Important methods:

length()

charAt()

substring()

equals()

equalsIgnoreCase()

compareTo()

contains()

indexOf()

toUpperCase()

toLowerCase()

trim()

replace()

split()

Example:

```
String s = "Hello World";

System.out.println(s.length());
```

```
System.out.println(s.substring(0,5));
System.out.println(s.toUpperCase());
```

4. StringBuilder and StringBuffer

Used for **mutable strings**.

Example:

```
StringBuilder sb = new StringBuilder("Hello");

sb.append(" Java");
System.out.println(sb);
```

Important methods:

- append()
- insert()
- delete()
- reverse()
- replace()

5. Array Operations

Very commonly used in NQT coding questions.

Declaration:

```
int[] arr = {3,1,5,2};
```

Access elements:

```
System.out.println(arr[0]);
```

Loop through array:

```
for(int i=0;i<arr.length;i++){
   System.out.println(arr[i]);
}
```

Enhanced for loop:

```
for(int x : arr){
   System.out.println(x);
}
```

6. Arrays Utility Class

Very important for **sorting and searching**.

Important methods:

- Arrays.sort()
- Arrays.binarySearch()

- Arrays.toString()
- Arrays.fill()

Example:

```
import java.util.Arrays;

int[] arr = {3,1,5,2};

Arrays.sort(arr);
System.out.println(Arrays.toString(arr));
```

7. Collections Framework (Very Important)

Most common collection classes asked in NQT.

ArrayList

Dynamic array.

```
import java.util.ArrayList;

ArrayList<Integer> list = new ArrayList<>();

list.add(10);
list.add(20);

System.out.println(list);
```

Important methods:

- add()
- remove()
- get()
- set()
- size()
- contains()

HashSet

Stores **unique elements**.

```
import java.util.HashSet;

HashSet<Integer> set = new HashSet<>();

set.add(1);
```

```
set.add(2);
set.add(2);

System.out.println(set);
```

HashMap

Stores **key-value pairs**.

```
import java.util.HashMap;

HashMap<String,Integer> map = new HashMap<>();
map.put("A",1);
map.put("B",2);

System.out.println(map.get("A"));
```

Important methods:

- put()
- get()
- containsKey()
- remove()
- keySet()

8. Exception Handling

Basic syntax.

```
try{
   int x = 10/0;
}
catch(ArithmeticException e){
   System.out.println("Error");
}
finally{
   System.out.println("Done");
}
```

Common exceptions:

- ArithmeticException
- NullPointerException
- ArrayIndexOutOfBoundsException
- NumberFormatException

9. Object Oriented Concepts (Very Important)

TCS frequently asks **OOP MCQs**.

Core concepts:

Encapsulation

```
class Student{
   private int id;

   public void setId(int id){
      this.id = id;
   }

   public int getId(){
      return id;
   }
}
```

Inheritance

```
class Animal{
   void sound(){
      System.out.println("Animal sound");
   }
}

class Dog extends Animal{
   void bark(){
      System.out.println("Dog barking");
   }
}
```

Polymorphism

Method overloading:

```
int add(int a,int b){
   return a+b;
}

int add(int a,int b,int c){
```

```
    return a+b+c;
}
```

Method overriding.

10. Java Sorting Tricks

Sorting collections with comparator.

Example:

```
import java.util.*;

ArrayList<Integer> list = new ArrayList<>();

list.add(5);
list.add(1);
list.add(3);

Collections.sort(list);

System.out.println(list);
```

Reverse sorting:

```
Collections.sort(list, Collections.reverseOrder());
```

11. Java Interview Tricks (Very Common MCQs)

Swap two numbers without temp.

```
a = a + b;
b = a - b;
a = a - b;
```

Reverse string.

```
String s = "hello";

String rev = new StringBuilder(s).reverse().toString();

System.out.println(rev);
```

Convert string to integer.

```
int num = Integer.parseInt("123");
```

12. Basic Algorithms

Revise Java implementation of:

- Binary Search
- Frequency Count using HashMap
- Two Pointer
- Sorting
- Array traversal

Example (frequency count):

```
HashMap<Integer,Integer> map = new HashMap<>();

for(int x : arr){
   map.put(x, map.getOrDefault(x,0)+1);
}
```

13. Time Complexity Basics

Know complexity of common operations.

Operation	Complexity
Array access	O(1)
ArrayList add	O(1)
HashMap lookup	O(1)
HashSet lookup	O(1)
Sorting	O(n log n)